DEVELOPPEMENT WEB

Chargé de cours

KOUASSI Née Kouakou Affoussatou

Professeur d'informatique de gestion

Année Scolaire 2025 - 2026

SOMMAIRE

CHAPITRE 1 : COMPRENDRE LE WEB 2
1) **DEFINITIONS ET HISTORIQUE** 2
2) **HISTORIQUE ET EVOLUTION DU WEB** 2
3) **FONCTIONNEMENT DU WEB** 2
4) **TECHNOLOGIES FONDAMENTALES DU WEB** 2
5) **LE DEVELOPPEMENT WEB** 2
CHAPITRE 2 : LE HTML 2

CHAPITRE 1 : COMPRENDRE LE WEB

1) DEFINITIONS ET HISTORIQUE

- **Internet** : Ensemble de réseaux mondiaux interconnectés qui permet à des ordinateurs et à des serveurs de communiquer efficacement au moyen d'un protocole de communication commun (IP). Ses principaux services sont le Web, le FTP, la messagerie et les groupes de discussion.
- **WEB**: Le web est l'abbréviation de World Wide Web. C'est un système hypertexte (c'est-à-dire qu'il permet d'établir des liaisons directes entre des éléments différents : texte, image, etc.), qui fonctionne sur le réseau internet. Les pages disponibles sur le web sont liées entre elles par des liens hypertextes

2) HISTORIQUE ET EVOLUTION DU WEB

- **1965** : Invention et programmation du concept d'hypertexte par Ted Nelson, qui permet de relier des pages entre elles.
- **1989** : Le World Wide Web (www) a été inventé par **Tim Berners-Lee**, un Britannique, dans un laboratoire du CERN (Centre européen de recherche nucléaire). World Wide Web est le premier navigateur web et le premier éditeur HTML.
- **1993** : Mise dans le domaine public, disponibilité du premier navigateur Mosaic. C'est la première fois que des internautes vont pouvoir se connecter sur le réseau mondial et ainsi échanger des informations.

Au cours de ces trente dernières années, certaines avancées ont permis une évolution exponentielle du développement du web. C'est ainsi que sont apparus de nouveaux langages qui ont permis aux développeurs de créer des sites dynamiques.

- **1995** : Mise à disposition de technologies pour le développement des premiers sites web interactifs (langage JavaScript) et dynamiques (langage PHP), avec l'apparition d'un langage de programmation qui augmente l'interactivité des pages web. Les sites web interactifs utilisent le langage JavaScript afin de créer des interactions entre les utilisateurs et le site.

La structure d'un site dynamique est codée en langage PHP (langage qui permet le dynamisme d'un site), à l'inverse d'un site statique qui est en général codé en HTML/CSS.

Les géants du web vont ainsi pouvoir créer leur propre navigateur grand public pour surfer sur la toile : Netscape, Microsoft Internet Explorer, Firefox, Chrome, Edge.

- **1998** : Apparition de Google, le premier moteur de recherche capable de révolutionner le web avec ses algorithmes extrêmement puissants.
- **1999** : Évolution du web en web 2.0.

L'interactivité permet désormais plus de simplicité d'utilisation (on peut échanger des messages entre le client et le serveur).

- **2001** : Standardisation des pages grâce au DOM (Document Object Model).

Il a fallu standardiser les documents que l'on ajoute au web, afin qu'ils soient visibles et consultables par tous en fonction des normes adoptées. Le DOM est le document modèle de cette standardisation.

- **2010** : Mise à disposition de technologies pour le développement d'applications sur mobiles.
- 2014 : Le nombre de sites créés sur le web dépasse le milliard.

Aujourd'hui : Nous sommes à l'ère du Web 3.0 (IA, blockchain, sémantique, services intelligents).

3) FONCTIONNEMENT DU WEB

Le Web fonctionne selon un modèle client-serveur : votre navigateur (le client) envoie une requête HTTP pour une page web à un serveur, qui renvoie les fichiers (HTML, CSS, images, etc.) pour que votre navigateur les affiche. Le processus comprend l'utilisation du DNS pour traduire l'URL en adresse IP, le protocole HTTP(S) pour la communication, et la transmission des données via la connexion Internet.

Les étapes du fonctionnement du Web :

1. Lancement de la requête : Vous tapez une adresse web (URL) ou cliquez sur un lien dans votre navigateur.

2. Traduction de l'adresse (DNS) : Le navigateur utilise le système de noms de domaine (DNS) pour convertir l'URL en une adresse IP numérique unique, qui est l'adresse de destination du serveur.
3. Communication client-serveur : Le navigateur envoie une requête (généralement en utilisant le protocole HTTP ou HTTPS) au serveur web identifié par l'adresse IP.
4. Traitement par le serveur : Le serveur reçoit la requête, localise la page ou les données demandées et prépare une réponse.
5. Envoi de la réponse : Le serveur renvoie les fichiers de la page web (le contenu HTML, les images, le CSS, le JavaScript, etc.) vers votre navigateur.
6. Affichage par le navigateur : Votre navigateur reçoit ces données, les traite et les assemble pour afficher la page web dans votre interface.



Fonctionnement du web

Client : L'appareil (ordinateur, smartphone) et le logiciel (navigateur web comme Chrome, Firefox) que vous utilisez pour accéder au Web.

Serveur : L'ordinateur qui héberge et stocke les sites web, les applications et les fichiers.

Protocoles : Des règles qui régissent la communication entre le client et le serveur, comme le protocole HTTP(S) qui permet de demander et d'envoyer des pages web.

DNS : Un système qui fait le lien entre les noms de domaine faciles à retenir (comme example.com) et les adresses IP numériques nécessaires pour trouver les serveurs.

Adresse IP : Une série de chiffres qui identifie un appareil spécifique sur le réseau, comme les autres ordinateurs, téléphones ou serveurs.

4) TECHNOLOGIES FONDAMENTALES DU WEB

a) Côté Client (Front-End)

Ces technologies sont gérées par le navigateur et sont directement visibles par l'utilisateur.

- **HTML** (HyperText Markup Language) : C'est le langage de base qui permet de structurer le contenu d'une page web, comme les titres, les paragraphes et les images.
- **CSS** (Cascading Style Sheets) : Utilisé pour le style, le design et la mise en page des éléments HTML, définissant l'apparence visuelle d'un site web
- **JavaScript** : Un langage de programmation qui ajoute de l'interactivité et des fonctionnalités dynamiques aux pages web, comme les animations ou les formulaires interactifs.

b) Côté Serveur (Back-End)

Ces technologies gèrent la logique d'une application web, le traitement des requêtes et l'interaction avec les bases de données.

- **Langages et Frameworks** : PHP (Symfony, Laravel), Python (Django, Flask), Java (Spring), JavaScript (Node.js), Ruby (Rails), C# (ASP.NET).

Les langages backend les plus populaires incluent Java, Python, et PHP, mais on trouve aussi JavaScript (Node.js), Ruby, Golang, et C#. Les frameworks comme Spring pour Java, Django pour Python, et Laravel/Symfony pour PHP fournissent des composants et des outils prêts à l'emploi pour accélérer le développement d'applications. Le choix dépend des besoins du projet.

- **Serveurs Web** : Apache, Nginx, IIS…

Un serveur web est un système (composé de matériel et de logiciel) qui héberge et diffuse les fichiers d'un site web ou d'une application web. Lorsqu'un utilisateur saisit une URL dans son navigateur, le serveur web reçoit une requête, trouve les données demandées (souvent en communiquant avec une base de données) et renvoie les informations, comme des pages HTML, des images ou d'autres fichiers, au navigateur pour qu'il les affiche.

- **Bases de données** : MySQL, PostgreSQL, Oracle, MongoDB…

Une base de données est un système informatique structuré permettant de collecter, stocker, organiser, gérer et récupérer des informations de manière efficace.

c) Protocoles et standards

Les protocoles sont des règles qui régissent la communication entre le client et le serveur.

Protocole	Définition	Rôle	Importance
TCP/IP (Transmission Control Protocol / Internet Protocol)	Ensemble de protocoles de communication qui définissent comment les données sont découpées, adressées, transmises et reconstituées sur un réseau.	- **IP** : identifie les machines grâce à une adresse unique. - **TCP** : assure la transmission fiable (contrôle d'erreurs, séquencement des paquets).	Base d'Internet et du Web : sans TCP/IP, aucune communication entre ordinateurs ne serait possible.
HTTP (HyperText Transfer Protocol)	Protocole de communication client-serveur utilisé pour transférer des pages web.	Transmet des requêtes (navigateur → serveur) et des réponses (serveur → navigateur).	Base du fonctionnement du Web, permet la consultation de sites internet.
HTTPS (HTTP Secure)	Version sécurisée de HTTP avec chiffrement (SSL/TLS).	Garantit la confidentialité et l'intégrité des échanges.	Indispensable pour protéger les données sensibles (paiement, identifiants).
URLs (Uniform Resource Locators)	Adresses uniques qui identifient les ressources (pages web, images, etc.) sur le Web.	Spécifient non seulement le nom de la ressource, mais aussi son emplacement sur le Web. Par exemple, *https://www.example.com/page.html*.	Améliorent la navigation et l'expérience utilisateur en rendant le contenu facile à trouver et à partager
DNS (Domain Name System)	Système de traduction des noms de domaine en adresses IP.	Transforme une URL lisible (ex. google.com) en adresse IP numérique.	Permet la navigation conviviale sans mémoriser d'adresses IP.
FTP (File Transfer Protocol)	Protocole de transfert de fichiers entre ordinateurs via Internet.	Permet de télécharger ou envoyer des fichiers sur un serveur.	Utilisé pour gérer et héberger des sites web (upload des pages, scripts).
SMTP (Simple Mail Transfer Protocol)	Protocole standard pour l'envoi d'e-mails.	Transmet les courriels d'un client de messagerie vers un serveur.	Base de la communication par email.
IMAP (Internet Message	Protocole de gestion des emails côté serveur.	Permet de consulter ses emails en restant synchronisé avec le serveur.	Idéal pour accéder aux emails sur plusieurs appareils.

Access Protocol)			
POP3 (Post Office Protocol v3)	Protocole de réception des emails.	Télécharge les emails du serveur vers l'appareil de l'utilisateur.	Ancien standard, utile pour consultation hors ligne mais moins flexible qu'IMAP.
SSL/TLS (Secure Socket Layer / Transport Layer Security)	Protocoles de chiffrement des communications.	Protègent les échanges entre client et serveur.	Assurent sécurité et confidentialité (banques, e-commerce).
WebSocket	Protocole de communication bidirectionnelle en temps réel.	Maintient une connexion ouverte entre client et serveur.	Utilisé pour les chats, jeux en ligne, applications temps réel.
SSH (Secure Shell)	Protocole sécurisé de communication à distance.	Permet d'administrer un serveur via une connexion chiffrée.	Indispensable pour la gestion sécurisée des serveurs Web.

5) LE DEVELOPPEMENT WEB

Le développement web est le processus de conception, création et maintenance de sites web et applications en ligne, impliquant la programmation, le design et la gestion des serveurs et bases de données. Il s'appuie sur des langages de programmation comme HTML, CSS, JavaScript et PHP pour structurer, mettre en forme et rendre interactives les pages web, et peut être divisé en développement front-end (côté client) et back-end (côté serveur).

Le développement web englobe toutes les étapes nécessaires pour qu'un contenu soit accessible sur Internet, depuis l'architecture et le codage jusqu'à l'hébergement sur des serveurs.

- La conception et le développement : Il s'agit de créer la structure et le contenu d'un site ou d'une application.
- La programmation : Les développeurs écrivent le code à l'aide de langages informatiques.
- La gestion des bases de données : Les informations sont stockées dans des bases de données et gérées par les développeurs.
- La gestion des serveurs : L'hébergement et la configuration des serveurs web font partie intégrante du processus.

a) Principales technologies :

HTML, CSS, JavaScript, PHP ou autres

b) Les types de développement :

Le développement web se divise généralement en deux spécialités :

- **Front-end** : Se concentre sur l'interface utilisateur (UI) et l'expérience utilisateur (UX), c'est-à-dire tout ce que le visiteur voit et avec quoi il interagit directement sur son navigateur.
- **Back-end** : Traite de la partie "invisible" du site, comme les serveurs, les bases de données et la logique applicative, et gère le fonctionnement général du site.
- **Full-stack** : Un développeur full-stack maîtrise à la fois les aspects front-end et back-end.

c) Environnement de développement web

Un environnement de développement web est l'ensemble des outils et logiciels nécessaires pour créer, tester et déployer des sites web et des applications web.

- **Editeur de code** : Un éditeur de code est un programme informatique utilisé par les développeurs pour écrire, modifier et organiser du code source pour créer des logiciels ou des sites web. Il offre des fonctionnalités avancées par rapport à un éditeur de texte classique, comme la coloration syntaxique qui met en évidence les différents éléments du code, l'autocomplétions pour une saisie plus rapide, ainsi que des outils de débogage et de gestion de projet. Parmi les éditeurs de code les plus populaires, on trouve Visual Studio Code, Sublime Text, Notepad++…

- **Navigateur web** : Un navigateur web est un logiciel qui permet d'accéder aux sites web et de les afficher sur un appareil comme un ordinateur, une tablette ou un smartphone. Il interprète le code des pages web pour les présenter à l'utilisateur. Parmi les navigateurs les plus populaires, on trouve Google Chrome, Safari, Microsoft Edge et Mozilla Firefox.

- **Serveur web local** : est un serveur installé sur votre ordinateur qui vous permet de tester et développer des applications et des sites web sans être connecté à Internet. Des outils comme les packs XAMPP, WAMP, ou MAMP sont des solutions populaires pour créer un environnement de serveur local en incluant des logiciels comme Apache (serveur web), MySQL/MariaDB (base de données) et PHP.

CHAPITRE 2 : CAHIER DES CHARGES D'UN PROJET WEB

1) DEFINITION

Le **cahier des charges** est un document contractuel qui décrit **de manière précise et détaillée** les besoins, les objectifs, les contraintes et les attentes d'un projet.

Il sert de **référence commune** entre :

- Le **commanditaire** (client)
- Le **prestataire** (développeur, agence ou équipe technique)

C'est la **feuille de route** du projet.

2) LES OBJECTIFS DU CAHIER DES CHARGES

- **Exprimer clairement les besoins du client** : Que doit faire le site ou l'application ?
- **Définir les fonctionnalités attendues** : Liste précise des fonctions à développer.
- **Fixer les contraintes techniques et budgétaires** : Hébergement, compatibilité, délais, coût.
- **Servir de base au contrat** : Document de référence pour évaluer le respect du projet.
- **Faciliter la communication** : Entre le client, le chef de projet, le graphiste, et le développeur.

3) STRUCTURE GENERALE D'UN CAHIER DES CHARGES WEB

a) Page de couverture

- Titre du projet
- Nom du client et du prestataire
- Date de rédaction
- Version du document

b) Présentation du projet

- Contexte général
- Objectifs du site ou de l'application
- Public cible
- Enjeux et bénéfices attendus

Exemple *: L'entreprise « Good Food » souhaite disposer d'un site web pour présenter ses produits et recevoir des commandes en ligne.*

c) Présentation du commanditaire

- Nom de l'entreprise
- Activités principales
- Effectif
- Adresse et contacts

d) Objectifs détaillés du projet

- Objectif principal
- Objectifs secondaires
- Résultats attendus (indicateurs mesurables)

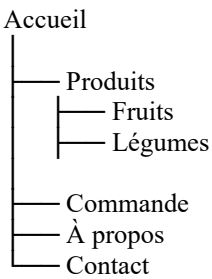
e) Description fonctionnelle (besoins du client)

Liste des **fonctions principales** du site, par module :

Module	Fonctionnalités attendues
Page d'accueil	Présentation de l'entreprise, navigation claire
Catalogue	Liste des produits avec photo, prix, description
Panier	Ajouter / supprimer un article
Commande	Formulaire de commande en ligne
Espace admin	Gestion des produits et des commandes

f) Architecture du site (arborescence)

Présente la **structure hiérarchique** des pages.
Exemple d'arborescence :



g) Charte graphique et design

- Logo, couleurs, typographies
- Exemples de mise en page
- Maquettes ou wireframes

h) Contraintes techniques

- Langages utilisés : HTML, CSS, JavaScript, PHP, MySQL

- Compatibilité navigateur : Chrome, Firefox, Safari
- Hébergement et nom de domaine
- Sécurité (HTTPS, validation de formulaires, etc.)

i) Contraintes organisationnelles

- Délai de livraison
- Budget estimé
- Planning prévisionnel
- Répartition des rôles dans l'équipe

j) Maintenance et évolutivité

- Suivi post-production
- Sauvegarde et mise à jour des données
- Prévision d'ajout futur de nouvelles fonctionnalités

4) METHODE DE REDACTION D'UN CAHIER DES CHARGES

1. **Recueillir les besoins du client** : Par entretiens, formulaires ou questionnaires…
2. **Analyser les besoins** : Distinguer besoins essentiels et besoins optionnels.
3. **Rédiger clairement et simplement** : Sans jargon technique inutile.
4. **Structurer le document** : Suivre un plan logique et cohérent.
5. **Faire valider par le client** : Avant de démarrer le développement.

NB :

- Impliquez le client dans toutes les étapes.
- Mettez à jour le cahier des charges à chaque modification majeure.
- Vérifiez la cohérence entre les besoins et les fonctionnalités finales.

CHAPITRE 3 : LE HTML

1) PRESENTATION

Le HTML est un langage de balisage utilisé pour créer la structure des pages web. Il est constitué de balises qui permettent d'indiquer aux navigateurs comment afficher le contenu.

Structure de base d'un document HTML :

Voici la structure de base d'un document HTML5 :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mon premier site</title>
</head>
<body>
    <h1>Bienvenue sur mon site !</h1>
    <p>Ceci est un paragraphe de texte.</p>
</body>
</html>
```

- ***<!DOCTYPE html>*** : Indique au navigateur qu'il s'agit d'un document HTML5.
- ***<html>*** : C'est la balise racine du document.
- ***<head>*** : Contient les métadonnées du document (comme le titre et les liens vers les fichiers CSS ou JavaScript).
- ***<meta charset="UTF-8">*** : Déclare l'encodage des caractères, ici UTF-8 (assure que les caractères spéciaux sont bien affichés).
- ***<title>*** : Déclare le titre de la page qui apparaîtra dans l'onglet du navigateur.
- ***<body>*** : Contient le contenu visible de la page web.

2) LES BALISES DE STRUCTURE DE CONTENU

Balise	Rôle
<header>	En-tête d'une section ou page
<nav>	Menu de navigation
<main>	Contenu principal de la page
<section>	Partie d'un contenu
<article>	Contenu autonome (ex: blog, news, forum)
<aside>	Contenu secondaire (ex: pub, infos liées)
<footer>	Pied de page ou de section
** **	Conteneur générique sans signification

3) LES BALISES DE TEXTE

Balise	Rôle
<h1> à <h6>	Titres de niveau 1 à 6
<p>	Paragraphe de texte
** **	Saut de ligne
<hr>	Ligne horizontale (séparation)
****	Texte important (souvent en gras)
****	Texte souligné ou en italique (emphase)
<blockquote>	Citation longue
<q>	Citation courte (inline)
<code>	Affiche du code informatique
<pre>	Préserve les espaces et retours à la ligne
****	Conteneur inline (utile pour styliser du texte)

4) LES BALISES DE LIEN ET DE NAVIGATION

Balise	Rôle
<a href="">	Crée un lien hypertexte vers une page ou ressource
target="_blank"	(Attribut) Ouvre le lien dans un nouvel onglet
<nav>	Contient des liens de navigation

```html
<nav>
  <a href="index.html">Accueil</a>
  <a href="contact.html">Contact</a>
</nav>
```

5) LES BALISES POUR LES MEDIAS

Balise	Rôle
<img src="" alt="">	Insère une image
<audio controls>	Intègre un fichier audio
<video controls>	Intègre une vidéo
<source>	Fournit différentes sources média
<figure>	Conteneur pour image + légende
<figcaption>	Légende d'une image ou d'un média

6) LES BALISES DE LISTE

Balise	Rôle
** **	Liste non ordonnée (puces)
** **	Liste ordonnée (numérotation)
** **	Élément de liste
<dl>	Liste de définition
<dt>	Terme à définir
<dd>	Définition du terme

7) LES BALISES DE TABLEAU

Balise	Rôle
<table>	Crée un tableau
<tr>	Crée une ligne
<td>	Cellule de données
<th>	Cellule d'en-tête (gras, centrée)
<thead>	En-tête du tableau

<tbody>	Corps du tableau
<tfoot>	Pied du tableau
colspan / rowspan	Fusion de colonnes ou lignes

8) BALISES DE FORMULAIRE

Balise	Rôle
<form>	Conteneur du formulaire
<input>	Champ de saisie
<textarea>	Zone de texte
<button>	Bouton
<select>	Liste déroulante
<option>	Option dans la liste déroulante
<label>	Étiquette liée à un champ
<fieldset>	Groupe de champs
<legend>	Légende pour un fieldset

9) **BALISES SEMANTIQUES** (HTML5)

Ces balises ajoutent de la **signification** au contenu, pour les navigateurs, moteurs de recherche et technologies d'assistance.

Balise	Rôle
<article>	Contenu indépendant
<section>	Regroupe un thème ou une partie du contenu
<aside>	Contenu secondaire (ex: barre latérale)
<header>	En-tête d'un bloc ou de page
<footer>	Pied d'un bloc ou de page
<main>	Contenu principal (unique sur la page)
<nav>	Navigation principale
<figure>	Conteneur d'une image ou graphique avec légende
<figcaption>	Légende associée à une figure

CHAPITRE 4 : LE CSS

1) DEFINITION

Le **CSS** (Cascading Style Sheets) est langage utilisé pour définir le style et la mise en forme des pages Web (HTML). Son rôle est de séparer le **contenu** (HTML) de la **présentation** (CSS).

2) LES METHODES D'INTEGRATION DU CSS AVEC LE HTML

a- Inline CSS

Cette méthode permet d'appliquer directement un style sur une balise via l'attribut **style**. Elle est peu recommandée sauf pour des tests rapides ou des styles très spécifiques.

Exemple :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Exemple CSS Inline</title>
</head>
<body>
    <h1 style="color: blue; text-align: center;">Bonjour le Web</h1>
    <p style="font-size: 18px; background-color: yellow;">Ceci est un paragraphe avec du CSS en ligne.</p>
</body>
</html>

```

b- Internal CSS

Cette méthode permet de placer le code CSS dans la section **<head>** du fichier HTML avec la balise **<style>.** Elle est utile pour des petites pages ou des tests.

Exemple :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Exemple CSS Interne</title>
    <style>
        body {
            background-color: #f0f0f0;
            font-family: Arial, sans-serif;
        }
        h1 {
            color: green;
            text-align: center;
        }
        p {
            font-size: 18px;
            color: darkred;
        }
    </style>
</head>
<body>
    <h1>Cours CSS</h1>
    <p>Ceci est un exemple de CSS interne.</p>
</body>
</html>
```

c- External CSS

Le style est écrit dans un fichier **.css** séparé et lié au HTML avec la balise **<link>.** C'est la méthode la plus propre et qui est utilisée en développement professionnel.

Exemple :

Fichier Html (index.html)

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Exemple CSS Externe</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Cours CSS</h1>
    <p>Ceci est un exemple de CSS externe.</p>
</body>
</html>
```

Fichier (style.css)

```css
body {
    background-color: #fafafa;
    font-family: Verdana, sans-serif;
}

h1 {
    color: purple;
    text-align: center;
}

p {
    font-size: 20px;
    color: navy;
}
```

3) LES CATEGORIES DE SELECTEURS CSS (LES "BALISES" CIBLEES)

a) **Sélecteurs de type (balises HTML)**

Ils ciblent directement une balise HTML.

Exemple :

```css
h1 {
  color: red;
}
```

Tous les titres <h1> s'afficheront en rouge.

b) **Sélecteurs de classe (.nom)**

Permettent d'appliquer un style à un groupe d'éléments.

Exemple :

```css
.paragraphe {
  font-size: 18px;
  color: green;
}
```

```
<p class="paragraphe">Texte avec classe CSS</p>
```

Seuls les paragraphes avec class="paragraphe" auront ce style.

c) **Sélecteurs d'identifiant (#id)**

Un identifiant unique appliqué à un seul élément.

Exemple :

```
#menu {
  background-color: lightgray;
  padding: 10px;
}
```

```
html

<div id="menu">Menu principal</div>
```

L'élément ayant id="menu" recevra ce style.

d) **Sélecteurs universels (*)**

Appliquent un style à tous les éléments de la page.

Exemple :

```
* {
  font-family: Arial, sans-serif;
}
```

Tout le texte de la page utilisera la police Arial.

e) Sélecteurs descendants (hiérarchiques)

Ciblent un élément en fonction de sa position dans la hiérarchie HTML.

Exemple :

```
css

div p {
  color: blue;
}
```

```html
<div>
  <p>Texte bleu</p>
</div>
```

Ici, seuls les <p> à l'intérieur d'un seront bleus.

f) **Sélecteurs d'attributs**

Ciblent un élément selon un attribut particulier.

Exemple :

```
input[type="text"] {
  border: 1px solid black;
}
```

Seuls les champs de type "texte" auront une bordure noire.

g) **Pseudo-classes**

Ciblent un état particulier d'un élément.

Exemples :

```
a:hover {
  color: red;
}
```

Quand on survole un lien, il devient rouge.

```
input:focus {
  border: 2px solid blue;
}
```

Quand un champ est actif, il reçoit une bordure bleue.

h) **Pseudo-éléments**

Permettent de cibler une partie d'un élément.

Exemple :

```
p::first-letter {
```

```
  font-size: 30px;
  color: purple;
}
```

La première lettre du paragraphe sera agrandie et colorée.

4) TABLEAU DES PRINCIPALES MANIPULATIONS EN CSS

Catégorie	Propriété CSS	Description / Rôle	Exemple
Texte & Police	color	Définit la couleur du texte	`p { color: blue; }`
	font-family	Choisit la police de caractères	`p { font-family: Arial, sans-serif; }`
	font-size	Définit la taille du texte	`h1 { font-size: 32px; }`
	font-weight	Définit l'épaisseur du texte (gras, normal…)	`h2 { font-weight: bold; }`
	text-align	Aligne le texte (gauche, droite, centre, justifié)	`p { text-align: justify; }`
	text-decoration	Ajoute soulignement, barré, etc.	`a { text-decoration: none; }`
	line-height	Hauteur de ligne	`p { line-height: 1.5; }`
Couleurs & Fonds	background-color	Couleur de fond d'un élément	`body { background-color: lightgray; }`
	background-image	Image de fond	`div { background-image: url("fond.png"); }`
	opacity	Définit la transparence	`img { opacity: 0.7; }`
Bordures & Ombres	border	Définit une bordure (taille, style, couleur)	`div { border: 2px solid black; }`
	border-radius	Arrondit les coins	`img { border-radius: 50%; }`
	box-shadow	Ajoute une ombre portée	`div { box-shadow: 2px 2px 5px gray; }`
Dimensions & Espacements	width / height	Définit largeur et hauteur	`div { width: 200px; height: 100px; }`
	margin	Marge extérieure	`p { margin: 20px; }`
	padding	Marge intérieure	`div { padding: 15px; }`
	max-width / min-width	Contraintes de taille	`img { max-width: 100%; }`
Mise en Page (Layout)	display	Mode d'affichage (block, inline, flex, grid…)	`div { display: flex; }`
	position	Positionnement (static, relative, absolute, fixed, sticky)	`header { position: fixed; top: 0; }`
	float	Aligne un élément à gauche/droite	`img { float: right; }`
	z-index	Ordre de superposition	`div { z-index: 10; }`

	flex	Gestion des conteneurs flexibles	`.container { display: flex; justify-content: space-between; }`
	grid	Mise en page en grille	`.grid { display: grid; grid-template-columns: 1fr 2fr; }`
Transitions & Animations	transition	Ajoute des effets progressifs	`div { transition: all 0.5s ease; }`
	transform	Transformation (rotation, échelle…)	`div { transform: rotate(45deg); }`
	animation	Définit une animation CSS	`div { animation: slide 3s infinite; }`
Liens & Listes	list-style	Personnalise les puces de liste	`ul { list-style: square; }`
	cursor	Change le pointeur de souris	`a { cursor: pointer; }`
Tableaux	border-collapse	Fusionne les bordures	`table { border-collapse: collapse; }`
	nth-child	Sélectionne une ligne/colonne précise	`tr:nth-child(even) { background: #f2f2f2; }`
Multimédia & Responsive	object-fit	Ajuste l'affichage des images/vidéos	`img { object-fit: cover; }`
	@media	Media queries pour responsive design	`@media (max-width: 600px) { body { font-size: 14px; } }`

CHAPITRE 5 : BACK-END ET SERVEUR WEB

En informatique, le back-end (ou backend) fait référence à la partie d'une application ou d'un système informatique qui n'est pas directement accessible aux utilisateurs finaux. Il comprend principalement la gestion des données, la logique métier, l'accès aux bases de données, et d'autres fonctionnalités nécessaires au bon fonctionnement de l'application.

1) Introduction aux langages de programmation côté serveur : PHP

PHP (Hypertext Preprocessor) est un langage de programmation côté serveur très populaire, principalement utilisé pour le développement web dynamique.
Les principaux aspects et fonctionnalités de PHP :

a) Origines et Caractéristiques

- Origines : PHP a été créé par **Rasmus Lerdorf** en 1994. Initialement, PHP signifiait « Personal Home Page (Pages Personnelles) », mais il est maintenant appelé « **Hypertext Preprocessor** ».
- Langage de Script Côté Serveur : PHP est principalement utilisé pour générer des pages web dynamiques à partir du serveur. Cela signifie qu'il est exécuté sur le serveur web avant que le contenu ne soit envoyé au navigateur de l'utilisateur.

b) Fonctionnalités Principales

- **Intégration HTML :** PHP peut être facilement intégré dans du code HTML, ce qui permet aux développeurs de combiner logique et présentation dans le même fichier.
- **Compatibilité avec de Nombreuses Bases de Données :** PHP supporte une large gamme de systèmes de gestion de bases de données (SGBD), ce qui en fait un choix polyvalent pour le développement web.
- **Large Communauté et Support :** PHP bénéficie d'une large communauté de développeurs et est bien documenté, avec de nombreux frameworks et bibliothèques disponibles pour faciliter le développement.

c)Installation de PHP et Configuration

ETAPE 1 : Installation via un serveur local

Pour exécuter des scripts PHP, il est nécessaire d'avoir un **serveur web** et un **interpréteur PHP**.

Les solutions les plus courantes pour créer un environnement local sont : **XAMPP**, **WAMP**, **LAMP** et **MAMP**.

1. XAMPP (Windows, Linux, macOS)

XAMPP est un paquet tout-en-un contenant **Apache (serveur web)**, **MySQL/MariaDB (base de données)**, **PHP** et **Perl**.

Étapes d'installation :

1. Télécharger XAMPP depuis https://www.apachefriends.org.
2. Installer le logiciel et ouvrir le panneau de contrôle XAMPP.
3. Démarrer les modules Apache et MySQL.
4. Placer les fichiers PHP dans le dossier `htdocs` (ex. : `C:\xampp\htdocs\mon_projet`).
5. Ouvrir un navigateur et taper :
6. `http://localhost/mon_projet`

2. WAMP (Windows uniquement)

WAMP signifie **Windows, Apache, MySQL, PHP**. Il est spécialement conçu pour Windows.

Étapes d'installation :

1. Télécharger WAMP depuis https://www.wampserver.com.
2. Installer le logiciel et lancer **WampServer**.
3. Une icône apparaît dans la barre des tâches (verte lorsque les services sont actifs).
4. Placer les fichiers PHP dans le dossier `www` (ex. : `C:\wamp64\www\site_test`).
5. Accéder au projet via :
6. `http://localhost/site_test`

3. LAMP (Linux uniquement)

LAMP regroupe **Linux, Apache, MySQL, PHP**.
Il est installé via le terminal.

Étapes d'installation :

```
sudo apt update
sudo apt install apache2 mysql-server php libapache2-mod-php php-mysql
```

- Placer les fichiers PHP dans le dossier `/var/www/html/`.
- Redémarrer le serveur Apache :

```
sudo systemctl restart apache2
```

- Accéder au site via : `http://localhost/`

4. MAMP (macOS et Windows)

MAMP (Macintosh, Apache, MySQL, PHP) est une solution clé en main pour **macOS** et **Windows**.

Elle permet de configurer facilement un environnement PHP local sans manipulation complexe.

Étapes d'installation :

1. Télécharger MAMP depuis https://www.mamp.info.
2. Installer le logiciel puis ouvrir **MAMP**.
3. Cliquer sur **Start Servers** pour démarrer **Apache** et **MySQL**.
4. Les fichiers de votre projet doivent être placés dans le dossier :

   `/Applications/MAMP/htdocs/` ou `C:\MAMP\htdocs\` sous Windows.
5. Ouvrir un navigateur et taper :

```
http://localhost:8888/nom_du_projet
```

ETAPE 2 : Vérification de l'installation

Créer un fichier nommé `info.php` dans le dossier du projet et saisissez le code suivant :

```
<?php
phpinfo();
?>
```

Puis ouvrir dans le navigateur :

```
http://localhost/info.php
```

Cette page affiche toutes les informations sur la configuration PHP de votre serveur local (version, extensions, paramètres, etc.).

2) Structure de base d'un fichier PHP

Un fichier PHP contient généralement un mélange de **code HTML** et de **code PHP**. Le code PHP est interprété par le serveur avant que la page ne soit envoyée au navigateur du client. L'extension des fichiers PHP est **`.php`**

a. Balises PHP

Le code PHP est toujours encadré par des balises d'ouverture et de fermeture :

```
<?php
   // Code PHP ici
?>
```

Tout ce qui se trouve **entre ces balises** est interprété comme du code PHP, tandis que le reste du fichier (souvent du HTML) est renvoyé tel quel au navigateur.

b. Exemple simple

Voici un exemple d'un fichier PHP qui affiche un message sur une page web :

```
<!DOCTYPE html>
<html>
<head>
    <title>Première page PHP</title>
</head>
<body>

    <h1>Bienvenue sur ma première page PHP !</h1>

    <?php
        echo "Bonjour tout le monde !";
    ?>

</body>
</html>
```

Dans cet exemple :

- La partie **HTML** définit la structure de la page.
- Le code **PHP** (entre `<?php ... ?>`) est exécuté sur le serveur, puis le résultat est envoyé au navigateur.

c. Les commentaires en PHP

Les commentaires servent à documenter le code sans l'exécuter. Il existe trois façons d'écrire un commentaire en PHP :

```php
<?php
// Ceci est un commentaire sur une seule ligne

# Autre style de commentaire sur une ligne

/*
   Ceci est un commentaire
   sur plusieurs lignes
*/
?>
```

d. Exécution d'un fichier PHP

Pour exécuter un fichier PHP :

1. Placer le fichier dans le dossier du serveur local (`htdocs`).
2. Lancer le serveur (XAMPP).
3. Ouvrir le navigateur et accéder à :
   `http://localhost/mon_projet/nom_du_fichier.php`

3) Les Variables et Types de Données en PHP

Les **variables** sont utilisées pour **stocker des valeurs** (texte, nombre, booléen, etc.) qui peuvent être manipulées dans un programme.

En PHP, une variable **commence toujours par le symbole** `$`, suivi du nom de la variable.

a. Déclaration et affectation de variables

En PHP, on déclare une variable en lui affectant une valeur avec le signe = :

```php
<?php
$nom = "Alice";
$age = 25;
$ville = "Abidjan";

echo "Bonjour, je m'appelle $nom, j'ai $age ans et j'habite à $ville.";
?>
```

Remarques :

- Les noms de variables sont sensibles à la casse (`$Nom` et `$nom` sont différents).
- Une variable n'a pas besoin d'être déclarée avant son utilisation.
- Les guillemets doubles `"` permettent l'interprétation des variables dans les chaînes de caractères, contrairement aux guillemets simples `'`.

b. Les règles de nommage des variables

Un nom de variable PHP doit :

1. Commencer par le symbole `$`.
2. Commencer par une lettre ou un underscore _.
3. Contenir uniquement des lettres, chiffres ou underscores.
4. Ne pas contenir d'espaces ni de caractères spéciaux.

Exemples valides :

```
$nom;
$_prenom;
$age2;
```

Exemples invalides :

```
$2nom;     // commence par un chiffre
$nom-util; // contient un tiret
```

c. Les types de données en PHP

PHP est un **langage faiblement typé**, ce qui signifie qu'il n'est pas nécessaire de déclarer le type d'une variable avant de l'utiliser. Les principaux types sont :

Type	Description	Exemple
String (chaîne de caractères)	Texte entouré de guillemets simples ou doubles	`$nom = "Alice";`
Integer (entier)	Nombre entier positif ou négatif	`$age = 25;`
Float (nombre décimal)	Nombre à virgule flottante	`$prix = 19.99;`
Boolean	Vrai ou faux	`$disponible = true;`
Array (tableau)	Liste de valeurs ou de paires clé/valeur	`$jours = ["Lundi", "Mardi"];`
Object	Instance d'une classe	`$voiture = new Car();`
NULL	Valeur nulle ou inexistante	`$variable = null;`

d. Conversion de type

Il est possible de forcer une conversion de type :

```
<?php
$nombre = "10";
$nombre = (int)$nombre;  // convertit en entier
?>
```

e. Fonctions utiles sur les types

Quelques fonctions très utilisées :

```
gettype($var);   // Renvoie le type de la variable
var_dump($var);  // Affiche le type et la valeur
is_int($var);    // Vérifie si la variable est un entier
is_string($var); // Vérifie si c'est une chaîne
```

f. La portée des variables en PHP

La **portée** (ou *scope*) d'une variable détermine **l'endroit du programme où elle peut être utilisée**.
En PHP, il existe **quatre portées principales** :

1. Variable locale

Une variable déclarée **à l'intérieur d'une fonction** n'est accessible **que dans cette fonction**.

```
<?php
function afficherNom() {
    $nom = "Alice"; // variable locale
    echo $nom;
}
afficherNom();
// echo $nom; // Erreur : variable inaccessible ici
?>
```

`$nom` n'existe que dans la fonction `afficherNom()`.

2. Variable globale

Une variable déclarée **en dehors de toute fonction** est **globale**.
Elle n'est pas accessible directement à l'intérieur des fonctions.
Pour l'utiliser à l'intérieur d'une fonction, il faut employer le mot-clé **`global`**.

```
<?php
$age = 25; // variable globale

function afficherAge() {
    global $age; // on rend $age accessible
    echo "J'ai $age ans.";
}

afficherAge();
?>
```

3. Variables superglobales

PHP met à disposition des **variables spéciales** accessibles **partout dans le script**, sans déclaration préalable.
Elles commencent toutes par un double underscore _.

Les plus courantes sont :

Variable	Description
$_GET	Données envoyées par l'URL
$_POST	Données envoyées via un formulaire
$_SESSION	Données de session utilisateur
$_COOKIE	Données stockées côté client
$_SERVER	Informations sur le serveur et la requête
$_FILES	Fichiers téléchargés via un formulaire
$_REQUEST	Combine `$_GET`, `$_POST`, `$_COOKIE`
$_ENV	Variables d'environnement

Exemple :

```
<?php
echo $_SERVER['HTTP_HOST']; // Affiche le nom du serveur
?>
```

4. Variable statique

Une variable **statique** conserve sa valeur **entre plusieurs appels d'une fonction**.
Elle est déclarée à l'aide du mot-clé **`static`**.

```
<?php
function compteur() {
    static $nb = 0; // initialisée une seule fois
    $nb++;
    echo $nb . "<br>";
}

compteur(); // 1
compteur(); // 2
compteur(); // 3
?>
```

Ici, `$nb` garde sa valeur entre les exécutions successives de la fonction.

4. Les Constantes en PHP

Une **constante** est une donnée **dont la valeur ne change pas pendant l'exécution** du programme.

Contrairement aux variables, **elles ne commencent pas par $** et sont **automatiquement globales**.

a. Déclaration d'une constante

Il existe deux manières de définir une constante en PHP :

1. Avec la fonction `define()`

```
<?php
define("PI", 3.14);
define("SITE_WEB", "www.exemple.com");

echo PI;           // Affiche 3.14
echo SITE_WEB;     // Affiche www.exemple.com
?>
```

Remarques :

- Le nom d'une constante est généralement en **majuscules**.
- Par convention, les mots sont séparés par des underscores (_).
- Une fois définie, la constante **ne peut pas être redéfinie**.

2. Avec le mot-clé `const`

Depuis PHP 5.3, on peut aussi créer une constante avec le mot-clé **`const`** :

```
<?php
const TAUX_TVA = 0.18;
echo TAUX_TVA;
?>
```

Différences entre `define()` et `const` :

Caractéristiques	`define()`	`const`
Peut être utilisé dans une fonction ou une boucle	Oui	Non
Compatible avec les expressions dynamiques	Oui	Non
Nécessite des guillemets pour le nom	Oui	Non

b. Les constantes magiques

PHP fournit un ensemble de **constantes prédéfinies** appelées **constantes magiques**, dont la valeur dépend du contexte d'exécution.

Constante	Description	Exemple
`__LINE__`	Numéro de la ligne courante	`echo __LINE__;`
`__FILE__`	Chemin complet du fichier	`echo __FILE__;`
`__DIR__`	Répertoire du fichier courant	`echo __DIR__;`
`__FUNCTION__`	Nom de la fonction courante	`echo __FUNCTION__;`
`__CLASS__`	Nom de la classe courante	`echo __CLASS__;`
`__METHOD__`	Nom de la méthode courante	`echo __METHOD__;`
`__NAMESPACE__`	Nom de l'espace de noms courant	`echo __NAMESPACE__;`

Exemple :

```
<?php
echo "Fichier : " . __FILE__ . "<br>";
echo "Ligne : " . __LINE__;
?>
```

c. Constantes globales et bonnes pratiques

- Les constantes sont **disponibles partout** dans le script, sans besoin du mot-clé global.
- Elles sont utilisées pour définir des **valeurs fixes** : paramètres de configuration, taux, chemins d'accès, etc.
- Par convention, leur nom est écrit en **MAJUSCULES** pour les distinguer clairement des variables.

Exemple :

```php
<?php
define("CHEMIN_IMAGES", "assets/images/");
define("VERSION_APP", "1.0.0");

echo "Dossier images : " . CHEMIN_IMAGES;
?>
```

5. Les Opérateurs en PHP

Les opérateurs permettent d'effectuer des opérations sur des variables et des valeurs.

PHP dispose de plusieurs catégories d'opérateurs : **arithmétiques**, **d'affectation**, **de comparaison**, **logiques**, **d'incrémentation/décrémentation**, **de concaténation**, et **ternaire**.

a. Les opérateurs arithmétiques

Ils servent à effectuer des calculs numériques.

Opérateur	Description	Exemple	Résultat
+	Addition	`$x + $y`	Somme
-	Soustraction	`$x - $y`	Différence
*	Multiplication	`$x * $y`	Produit
/	Division	`$x / $y`	Quotient
%	Modulo (reste)	`$x % $y`	Reste de la division
**	Exponentiation	`$x ** $y`	x^y (x puissance y)

Exemple :

```php
<?php
$a = 10;
$b = 3;
echo $a + $b; // 13
echo $a % $b; // 1
?>
```

b. Les opérateurs d'affectation

Ils permettent d'attribuer une valeur à une variable.

Le plus simple est « = », mais PHP offre aussi des formes composées :

Opérateur	Exemple	Équivalent à	Résultat
=	$x = 5		5
+=	$x += 3	$x = $x + 3	Ajoute
-=	$x -= 2	$x = $x - 2	Soustrait
*=	$x *= 4	$x = $x * 4	Multiplie
/=	$x /= 2	$x = $x / 2	Divise
%=	$x %= 3	$x = $x % 3	Reste

c. Les opérateurs de comparaison

Ils comparent deux valeurs et renvoient **true** ou **false**.

Opérateur	Signification	Exemple	Résultat
==	Égalité (valeurs égales)	$x == $y	Vrai si valeurs identiques
===	Identité (valeur et type égaux)	$x === $y	Vrai si même type et valeur
!= ou <>	Différent de	$x != $y	Vrai si valeurs différentes
!==	Non identique	$x !== $y	Vrai si type ou valeur différent
>	Supérieur à	$x > $y	Vrai si $x > $y
<	Inférieur à	$x < $y	Vrai si $x < $y
>=	Supérieur ou égal	$x >= $y	Vrai si $x ≥ $y
<=	Inférieur ou égal	$x <= $y	Vrai si $x ≤ $y
<=>	Opérateur de comparaison combiné (spaceship)	$x <=> $y	Retourne **1** si $x est plus grand que $y, **0** s'ils sont égaux, et **-1** si $x est plus petits que $y

Exemple :

```
<?php
$a = 10;
$b = "10";
var_dump($a == $b);  // true
var_dump($a === $b); // false
?>
```

d. Les opérateurs logiques

Ils permettent de combiner plusieurs conditions.

Opérateur	Signification	Exemple	Résultat
and ou &&	ET logique	`$x && $y`	Vrai si les deux sont vrais
or ou `		`` ` ``	OU logique
!	NON logique	`!$x`	Inverse la valeur (true devient false)
xor	OU exclusif	`$x xor $y`	Vrai si un seul est vrai

Exemple :

```
<?php
$age = 20;
$carte = true;

if ($age >= 18 && $carte) {
    echo "Accès autorisé";
} else {
    echo "Accès refusé";
}
?>
```

e. Les opérateurs d'incrémentation et de décrémentation

Ils servent à augmenter ou diminuer la valeur d'une variable numérique.

Opérateur	Description	Exemple	Résultat
++$x	Pré-incrémentation	`$x = 5; ++$x;`	$x = 6 (puis retourne 6)
$x++	Post-incrémentation	`$x = 5; $x++;`	$x = 6 (mais retourne 5)
--$x	Pré-décrémentation	`$x = 5; --$x;`	$x = 4
$x--	Post-décrémentation	`$x = 5; $x--;`	$x = 4 (mais retourne 5)

f. L'opérateur de concaténation

Il sert à **assembler des chaînes de caractères**.

Opérateur	Exemple	Résultat
`.`	`$nom . " " . $prenom`	Fusionne les chaînes
`.=`	`$message .= "!";`	Ajoute du texte à la fin

Exemple :

```
<?php
$prenom = "Alice";
$nom = "Kouadio";
$identite = $prenom . " " . $nom;
echo $identite; // Affiche : Alice Kouadio
?>
```

g. L'opérateur ternaire

C'est une forme simplifiée du if...else.

```
<?php
$age = 18;
$message = ($age >= 18) ? "Majeur" : "Mineur";
echo $message;
?>
```

Équivalent à :

```
if ($age >= 18) {
    $message = "Majeur";
} else {
    $message = "Mineur";
}
```

6. Les structures conditionnelles en PHP

Les **structures conditionnelles** permettent d'exécuter une ou plusieurs instructions **en fonction d'une condition** (c'est-à-dire si une expression est vraie ou fausse).

Elles permettent donc d'ajouter de la logique et du dynamisme aux programmes PHP.

a. L'instruction if

La structure la plus simple est le **if**.

Elle permet d'exécuter un bloc de code **si et seulement si une condition est vraie**.

```
<?php
$age = 18;

if ($age >= 18) {
    echo "Vous êtes majeur.";
}
?>
```

b. L'instruction if ... else

Elle permet d'exécuter une action **si la condition est vraie**, et une autre **si elle est fausse**.

```
<?php
$age = 16;

if ($age >= 18) {
    echo "Vous êtes majeur.";
} else {
    echo "Vous êtes mineur.";
}
?>
```

c. L'instruction if ... elseif ... else

Elle permet de tester **plusieurs conditions successives**.

```
<?php
$note = 14;

if ($note >= 16) {
    echo "Très bien";
} elseif ($note >= 12) {
    echo "Assez bien";
} elseif ($note >= 10) {
    echo "Passable";
} else {
    echo "Insuffisant";
}
?>
```

d. La structure switch

La structure switch est utilisée lorsqu'on veut comparer une variable à plusieurs valeurs possibles.

C'est une alternative plus lisible à if...elseif...else.

```
<?php
$jour = "mardi";

switch ($jour) {
    case "lundi":
        echo "Début de semaine";
        break;
    case "mardi":
    case "mercredi":
    case "jeudi":
        echo "Milieu de semaine";
        break;
```

```
    case "vendredi":
        echo "Bientôt le week-end !";
        break;
    default:
        echo "C'est le week-end !";
}
?>
```

Remarque : L'instruction **break** permet de sortir du **switch** après l'exécution d'un cas.

Si on l'oublie, PHP continue d'exécuter les instructions des cas suivants.

7. Les boucles en PHP

Les **boucles** permettent de répéter une ou plusieurs instructions **tant qu'une condition est vraie** ou **pour un nombre déterminé de fois**.
Elles sont très utiles pour parcourir des tableaux, générer du contenu dynamique ou automatiser des tâches répétitives.

a. La boucle while

La boucle `while` exécute un bloc d'instructions **tant que la condition est vraie**.

```
<?php
$compteur = 1;

while ($compteur <= 5) {
    echo "Compteur : $compteur <br>";
    $compteur++;
}
?>
```

Ici, la boucle s'exécute 5 fois.
Quand `$compteur` devient supérieur à 5, la condition devient fausse et la boucle s'arrête.

b. La boucle do ... while

Contrairement à `while`, la boucle `do ... while` exécute **au moins une fois** le bloc d'instructions,
puis **vérifie la condition à la fin**.

```
<?php
$compteur = 1;

do {
    echo "Compteur : $compteur <br>";
    $compteur++;
```

```
} while ($compteur <= 5);
?>
```

Cette boucle garantit **au moins une exécution**, même si la condition est fausse dès le départ.

c. La boucle for

La boucle `for` est utilisée lorsque le **nombre d'itérations est connu à l'avance**.
Elle se compose de trois parties :

1. Initialisation
2. Condition
3. Incrémentation

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Tour n° $i <br>";
}
?>
```

Ici, `$i` commence à 1 et est incrémenté jusqu'à 5.
La boucle s'exécute donc 5 fois.

d. La boucle foreach

La boucle `foreach` est utilisée **pour parcourir les tableaux**.
Elle permet de récupérer facilement les **valeurs** (et éventuellement les **clés**) des éléments d'un tableau.

Exemple 1 : Parcourir les valeurs

```
<?php
$fruits = ["Pomme", "Banane", "Mangue"];

foreach ($fruits as $fruit) {
    echo "$fruit <br>";
}
?>
```

Exemple 2 : Parcourir les clés et les valeurs

```
<?php
$personne = ["nom" => "Kouakou", "prenom" => "Afsa", "age" => 25];

foreach ($personne as $cle => $valeur) {
    echo "$cle : $valeur <br>";
}
?>
```

e. Les instructions break et continue

break

Permet d'interrompre **immédiatement** l'exécution d'une boucle.

```
<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i == 5) break;
    echo "Valeur : $i <br>";
}
?>
```

La boucle s'arrête lorsque `$i` atteint 5.

continue

Permet d'**ignorer l'itération courante** et de passer à la suivante.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    if ($i == 3) continue;
    echo "Valeur : $i <br>";
}
?>
```

Ici, la valeur `3` n'est pas affichée.

f. Exemple combiné

```
<?php

for ($i = 1; $i <= 10; $i++) {
    if ($i % 2 == 0) {
        echo "$i est pair <br>";
    } else {
        echo "$i est impair <br>";
    }
}

?>
```

8. Les fonctions en PHP

Une **fonction** est un **bloc de code réutilisable** qui exécute une tâche précise.
Elle permet d'éviter la répétition du code, de rendre un programme plus clair, plus lisible et plus facile à maintenir.

a. Définition d'une fonction

En PHP, une fonction se déclare à l'aide du mot-clé `function` :

```
<?php

function direBonjour() {
    echo "Bonjour tout le monde !";
}
?>
```

Ici, la fonction `direBonjour()` ne s'exécute pas automatiquement.
Pour l'utiliser, il faut **l'appeler**.

b. Appel d'une fonction

Pour exécuter une fonction, il suffit de l'appeler par son nom suivi de parenthèses :

```
<?php
function direBonjour() {
    echo "Bonjour tout le monde !";
}

direBonjour();  // Appel de la fonction
?>
```

Le résultat affiché sera :
```
Bonjour tout le monde !
```

c. Fonctions avec paramètres

Une fonction peut recevoir des **paramètres** (ou arguments) qui lui permettent de travailler avec des valeurs variables.

```
<?php
function saluer($prenom) {
    echo "Bonjour $prenom !<br>";
}

saluer("Alain");
saluer("JEAN");
?>
```

Ici, la fonction affiche un message personnalisé selon le prénom passé en paramètre.

d. Valeur par défaut d'un paramètre

On peut définir une **valeur par défaut** pour un paramètre.
Si aucun argument n'est fourni lors de l'appel, la valeur par défaut sera utilisée.

```
<?php
function saluer($prenom = "visiteur") {
    echo "Bonjour $prenom !<br>";
}

saluer();           // Bonjour visiteur !
saluer("Amira");    // Bonjour Amira !
?>
```

e. Retour de valeur

Une fonction peut **renvoyer une valeur** à l'aide du mot-clé `return`.

```
<?php
function addition($a, $b) {
    return $a + $b;
}

$resultat = addition(5, 7);
echo "Le résultat est : $resultat";
?>
```

Ici, la fonction calcule la somme de `$a` et `$b`, puis renvoie le résultat.

f. Fonctions avec plusieurs paramètres

Une fonction peut accepter plusieurs paramètres :

```
<?php
function afficherInfos($nom, $age) {
    echo "Nom : $nom, Âge : $age ans<br>";
}

afficherInfos("Kouakou", 25);
afficherInfos("Diabaté", 30);
?>
```

g. La portée des variables (rappel)

En PHP, les variables ont une **portée**, c'est-à-dire une zone où elles sont accessibles :

Type de portée	Description
Locale	Variable déclarée à l'intérieur d'une fonction. Accessible uniquement dans cette fonction.
Globale	Variable déclarée à l'extérieur de toute fonction. Accessible partout sauf à l'intérieur des fonctions (sauf si on la rend globale).

Exemple 1 : portée locale

```
<?php
function test() {
    $x = 10; // variable locale
    echo "x vaut $x<br>";
}
test();
// echo $x; // Erreur : $x n'existe pas ici
?>
```

Exemple 2 : utiliser une variable globale

```
<?php
$x = 5; // variable globale

function test() {
    global $x; // rend accessible la variable globale $x
    echo "x vaut $x<br>";
}

test();
?>
```

Exemple 3 : superglobale $GLOBALS

La variable superglobale `$GLOBALS` en PHP est un tableau associatif qui contient toutes les variables globales définies dans un script. Elle est accessible dans tous les contextes, y compris les fonctions et les méthodes, sans avoir besoin de les déclarer avec global $variable. `$GLOBALS` permet de faire référence aux variables globales existantes en utilisant le nom de la variable comme index du tableau (par exemple, `$GLOBALS['ma_variable']`).

```
<?php
$x = 10;
$y = 20;

function additionner() {
    $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
}

additionner();
echo $z; // Affiche 30
?>
```

h. Fonctions intégrées (pré-définies) en PHP

PHP offre des **centaines de fonctions prédéfinies**.
Quelques exemples utiles :

Domaine	Exemple de fonction	Description
Chaînes de caractères	`strlen("texte")`	Longueur d'une chaîne
	`strtoupper("bonjour")`	Convertit en majuscules
	`strtolower("BONJOUR")`	Convertit en minuscules
Mathématiques	`abs(-5)`	Valeur absolue
	`sqrt(9)`	Racine carrée
	`rand(1,10)`	Nombre aléatoire
Tableaux	`count($tab)`	Nombre d'éléments
	`array_sum($tab)`	Somme des valeurs
	`in_array("valeur", $tab)`	Vérifie si une valeur existe

i. Fonctions anonymes (ou closures)

PHP permet de créer des fonctions **sans nom**, stockées dans des variables :

```
<?php
$saluer = function($nom) {
    return "Bonjour $nom !";
};

echo $saluer("Olivier");
?>
```

Ce type de fonction est très utilisé dans la programmation moderne (notamment avec les callbacks).

9. Les tableaux en PHP

Un **tableau** est une **structure de données** qui permet de stocker plusieurs valeurs dans une **seule variable**.
Chaque valeur est associée à une **clé** (un indice numérique ou une étiquette textuelle).

Les tableaux sont très utilisés en PHP pour manipuler des listes, des ensembles d'éléments ou des données complexes.

a. Déclaration d'un tableau

- *Tableau indexé (indices numériques)*

Les **tableaux indexés** utilisent des indices numériques automatiques à partir de 0.

```php
<?php
$fruits = ["Pomme", "Banane", "Mangue"];
// ou
$fruits = array("Pomme", "Banane", "Mangue");

echo $fruits[0]; // Affiche Pomme
echo $fruits[2]; // Affiche Mangue
?>
```

Ici, les indices sont :
`0 → Pomme, 1 → Banane, 2 → Mangue`

- *Tableau associatif*

Les **tableaux associatifs** utilisent des clés textuelles (chaînes de caractères).

```php
<?php
$personne = [
    "nom" => "Kamenan",
    "prenom" => "Olivier",
    "age" => 25
];

echo $personne["prenom"]; // Affiche Olivier
?>
```

Chaque clé représente une propriété descriptive.

- *Tableau multidimensionnel*

Les **tableaux multidimensionnels** contiennent d'autres tableaux à l'intérieur.
Ils sont utilisés pour représenter des données complexes (par ex. une liste d'utilisateurs).

```php
<?php

$utilisateurs = [
    ["nom" => "Kouakou", "prenom" => "Jean", "age" => 25],
    ["nom" => "Diabaté", "prenom" => "Amira", "age" => 30],
    ["nom" => "Koné", "prenom" => "Yao", "age" => 22]
];

echo $utilisateurs[1]["nom"]; // Affiche Diabaté
?>
```

Ici, `$utilisateurs[1]` correspond au deuxième utilisateur du tableau.

b. Ajout, modification et suppression d'éléments

- ***Ajouter un élément à un tableau indexé***

```php
<?php
$fruits = ["Pomme", "Banane"];
$fruits[] = "Mangue"; // Ajout à la fin du tableau
?>
```

- ***Modifier un élément***

```
$fruits[0] = "Orange";
```

- *Supprimer un élément*

```
unset($fruits[1]); // Supprime l'élément d'indice 1
```

c. Parcourir un tableau

- ***Avec une boucle for***

```
<?php
$fruits = ["Pomme", "Banane", "Mangue"];
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i] . "<br>";
}
?>
```

- ***Avec une boucle foreach***

```
<?php
$fruits = ["Pomme", "Banane", "Mangue"];
foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
?>
```

- ***Pour un tableau associatif***

```
<?php
$personne = ["nom" => "Koua", "prenom" => "Eva", "age" => 25];

foreach ($personne as $cle => $valeur) {
    echo "$cle : $valeur <br>";
}
?>
```

d. Fonctions utiles sur les tableaux

PHP fournit de nombreuses fonctions pour manipuler les tableaux :

Fonction	Description	Exemple
`count($tab)`	Nombre d'éléments	`count($fruits)`
`array_push($tab, $val)`	Ajoute un élément à la fin	`array_push($fruits, "Papaye")`
`array_pop($tab)`	Supprime le dernier élément	`array_pop($fruits)`
`array_shift($tab)`	Supprime le premier élément	`array_shift($fruits)`

array_unshift($tab, $val)	Ajoute un élément au début	`array_unshift($fruits, "Fraise")`
in_array($val, $tab)	Vérifie si une valeur existe	`in_array("Banane", $fruits)`
array_keys($tab)	Retourne toutes les clés	`array_keys($personne)`
array_values($tab)	Retourne toutes les valeurs	`array_values($personne)`
sort($tab)	Trie les valeurs (ordre croissant)	`sort($fruits)`
rsort($tab)	Trie les valeurs (ordre décroissant)	`rsort($fruits)`
asort($tab)	Trie un tableau associatif selon les valeurs	`asort($personne)`
ksort($tab)	Trie un tableau associatif selon les clés	`ksort($personne)`
implode(",", $tab)	Convertit un tableau en chaîne	`implode(", ", $fruits)`
explode(",", $chaine)	Convertit une chaîne en tableau	`explode(", ", "Pomme,Banane,Mangue")`

e. Exemple pratique

```
<?php
$etudiants = [
    ["nom" => "Kouadio", "note" => 15],
    ["nom" => "Diomandé", "note" => 13],
    ["nom" => "N'Guessan", "note" => 18]
];

$total = 0;
foreach ($etudiants as $etudiant) {
    echo $etudiant["nom"] . " : " . $etudiant["note"] . "<br>";
    $total += $etudiant["note"];
}

$moyenne = $total / count($etudiants);
echo "<br>Note moyenne : $moyenne";
?>
```

Résultat :

```
Kouadio : 15
Diomandé : 13
N'Guessan : 18

Note moyenne : 15.333333
```

10. Les formulaires en PHP

Les **formulaires HTML** permettent aux utilisateurs de saisir des données (texte, mot de passe, choix, etc.).
Ces données peuvent ensuite être **récupérées et traitées** par un script PHP.

PHP met à disposition deux superglobales principales pour accéder aux données d'un formulaire :

- **`$_GET`** : données envoyées par l'URL.
- **`$_POST`** : données envoyées de manière cachée (plus sécurisée).

a. Création d'un formulaire HTML

Exemple simple :

```
<!DOCTYPE html>
<html>
<head>
    <title>Formulaire PHP</title>
</head>
<body>

<form action="traitement.php" method="POST">
    <label>Nom :</label>
    <input type="text" name="nom"><br><br>

    <label>Âge :</label>
    <input type="number" name="age"><br><br>

    <input type="submit" value="Envoyer">
</form>

</body>
</html>
```

- L'attribut `action` indique le fichier PHP qui traitera les données.
- L'attribut `method` indique la méthode d'envoi (`POST` ou `GET`).

b. Récupération des données avec `$_POST`

Dans le fichier **traitement.php**, on peut récupérer les données ainsi :

```
<?php
$nom = $_POST["nom"];
$age = $_POST["age"];

echo "Bonjour $nom, vous avez $age ans.";
?>
```

La superglobale `$_POST` est un **tableau associatif** contenant les données envoyées via la méthode `POST`.

c. Récupération des données avec `$_GET`

Si le formulaire est envoyé avec la méthode `GET`, les données apparaissent dans l'URL :

```
<form action="traitement.php" method="GET">
```

Et dans **traitement.php** :

```
<?php
$nom = $_GET["nom"];
$age = $_GET["age"];

echo "Bonjour $nom, vous avez $age ans.";
?>
```

Exemple d'URL :
`http://localhost/traitement.php?nom=Afsa&age=25`

d. Différence entre $_GET et $_POST

Critère	$_GET	$_POST
Transmission	Par l'URL	Cachée (dans la requête HTTP)
Sécurité	Moins sécurisée	Plus sécurisée
Taille maximale	Limitée (~2000 caractères)	Aucune limite importante
Utilisation typique	Requêtes simples, liens	Formulaires sensibles (mots de passe, connexion)

e. Sécurisation des données reçues

Avant d'utiliser les données saisies par l'utilisateur, il faut **les sécuriser** pour éviter les failles (injection de code, XSS, etc.).

Utilisation de htmlspecialchars() :

```
<?php
$nom = htmlspecialchars($_POST["nom"]);
$age = htmlspecialchars($_POST["age"]);

echo "Bonjour $nom, vous avez $age ans.";
?>
```

Cette fonction empêche l'exécution de code HTML ou JavaScript injecté dans les champs.

f. Vérification de l'envoi du formulaire

Avant d'accéder aux variables, on vérifie si le formulaire a été soumis avec `isset()` ou `!empty()`.

```
<?php
if (isset($_POST["nom"]) && isset($_POST["age"])) {
    $nom = htmlspecialchars($_POST["nom"]);
    $age = htmlspecialchars($_POST["age"]);
```

```
    echo "Bonjour $nom, vous avez $age ans.";
} else {
    echo "Veuillez remplir le formulaire.";
}
?>
```

g. Exemple pratique complet

- *fichier : formulaire.php*

```
<!DOCTYPE html>
<html>
<head>
    <title>Formulaire complet</title>
</head>
<body>

<h2>Inscription</h2>
<form action="traitement.php" method="POST">
    <label>Nom :</label>
    <input type="text" name="nom" required><br><br>

    <label>Email :</label>
    <input type="email" name="email" required><br><br>

    <label>Mot de passe :</label>
    <input type="password" name="motdepasse" required><br><br>

    <input type="submit" value="S'inscrire">
</form>

</body>
</html>
```

- *fichier : traitement.php*

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $nom = htmlspecialchars($_POST["nom"]);
    $email = htmlspecialchars($_POST["email"]);
    $mdp = htmlspecialchars($_POST["motdepasse"]);

    echo "<h3>Vos informations :</h3>";
    echo "Nom : $nom <br>";
    echo "Email : $email <br>";
    echo "Mot de passe : $mdp";
}
?>
```

h. La superglobale $_SERVER

PHP fournit également la variable `$_SERVER`, qui contient des informations sur le serveur et la requête en cours.

Exemples :

```
<?php
echo $_SERVER["PHP_SELF"];  // Nom du fichier en cours
echo $_SERVER["SERVER_NAME"];  // Nom du serveur
echo $_SERVER["HTTP_USER_AGENT"];  // Navigateur du client
echo $_SERVER["REMOTE_ADDR"];  // Adresse IP du client
?>
```

CHAPITRE 6 : PHP ET MYSQL

1. Introduction : PHP et les bases de données

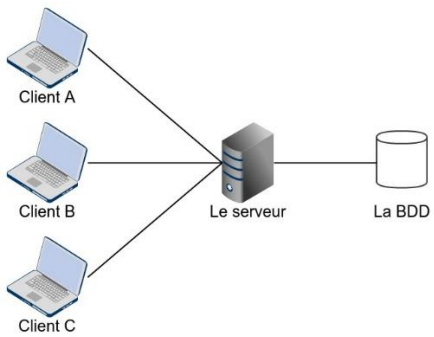
a. Rôle de MySQL

MySQL est un **Système de Gestion de Base de Données (SGBD)** relationnel. Il permet de **stocker, organiser et manipuler des données** sous forme de **tables**. MySQL est souvent utilisé avec PHP pour créer des sites dynamiques.

Exemples d'utilisation :

- Gestion des utilisateurs (authentification, profils)
- Enregistrement d'articles, produits, commandes, etc.

b. Architecture client-serveur



Le navigateur envoie une requête → le serveur PHP la traite → interagit avec MySQL → renvoie une réponse (HTML, JSON…).

2. Les deux principales extensions de connexion MySQL en PHP

Extension	Description	Avantages	Limites
MySQLi	Interface procédurale et orientée objet pour MySQL uniquement	Simple, rapide	Ne fonctionne qu'avec MySQL
PDO (PHP Data Objects)	Interface orientée objet pour plusieurs SGBD (MySQL, PostgreSQL, Oracle...)	Sécurisée, portable, moderne	Plus complexe au départ

Aujourd'hui, on privilégie **PDO** pour sa **portabilité** et sa **sécurité**.

a. Créer une base de données MySQL « gestion_etudiants »

```
CREATE DATABASE gestion_etudiants;
USE gestion_etudiants;
```

```
CREATE TABLE etudiants (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50),
    prenom VARCHAR(50),
    age INT,
    email VARCHAR(100)
);
```

3. Connexion base de données avec l'extension PHP « MySQLi »

```
<?php
$serveur = "localhost";
$utilisateur = "root";
$motdepasse = "";
$base = "gestion_etudiants";

$connexion = mysqli_connect($serveur, $utilisateur, $motdepasse, $base);

if (!$connexion) {
    die("Erreur de connexion : " . mysqli_connect_error());
}

echo "Connexion réussie à la base de données !";
?>
```

a. Insérer des données dans une table

Formulaire HTML :

```
<form action="ajouter.php" method="post">
    Nom : <input type="text" name="nom"><br>
    Prénom : <input type="text" name="prenom"><br>
    Âge : <input type="number" name="age"><br>
    Email : <input type="email" name="email"><br>
    <input type="submit" value="Ajouter">
</form>
```

Fichier `ajouter.php` :

```
<?php
include("connexion.php");

$nom = $_POST['nom'];
$prenom = $_POST['prenom'];
$age = $_POST['age'];
$email = $_POST['email'];

$requete = "INSERT INTO etudiants (nom, prenom, age, email)
            VALUES ('$nom', '$prenom', '$age', '$email')";

if (mysqli_query($connexion, $requete)) {
    echo "Étudiant ajouté avec succès !";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}
```

```
?>
```

b. Afficher les données de la base

```
<?php
include("connexion.php");

$requete = "SELECT * FROM etudiants";
$resultat = mysqli_query($connexion, $requete);

echo "<h2>Liste des étudiants</h2>";
echo "<table border='1'>
<tr><th>ID</th><th>Nom</th><th>Prénom</th><th>Âge</th><th>Email</th></tr>";

while ($ligne = mysqli_fetch_assoc($resultat)) {
    echo "<tr>
        <td>".$ligne['id']."</td>
        <td>".$ligne['nom']."</td>
        <td>".$ligne['prenom']."</td>
        <td>".$ligne['age']."</td>
        <td>".$ligne['email']."</td>
    </tr>";
}

echo "</table>";
?>
```

c. Modifier une donnée

Étape 1 : Créer un lien “Modifier”

```
<a href="modifier.php?id=<?php echo $ligne['id']; ?>">Modifier</a>
```

Étape 2 : Page `modifier.php`

```
<?php
include("connexion.php");
$id = $_GET['id'];
$result = mysqli_query($connexion, "SELECT * FROM etudiants WHERE id=$id");
$etudiant = mysqli_fetch_assoc($result);
?>

<form action="update.php" method="post">
    <input type="hidden" name="id" value="<?php echo $etudiant['id']; ?>">
    Nom : <input type="text" name="nom" value="<?php echo $etudiant['nom'];
?>"><br>
    Prénom : <input type="text" name="prenom" value="<?php echo
$etudiant['prenom']; ?>"><br>
    Âge : <input type="number" name="age" value="<?php echo
$etudiant['age']; ?>"><br>
    Email : <input type="email" name="email" value="<?php echo
$etudiant['email']; ?>"><br>
    <input type="submit" value="Mettre à jour">
</form>
```

Étape 3 : Page `update.php`

```
<?php
include("connexion.php");
```

```
$id = $_POST['id'];
$nom = $_POST['nom'];
$prenom = $_POST['prenom'];
$age = $_POST['age'];
$email = $_POST['email'];

$requete = "UPDATE etudiants SET nom='$nom', prenom='$prenom', age='$age',
email='$email' WHERE id=$id";

if (mysqli_query($connexion, $requete)) {
    echo "Mise à jour réussie !";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}
?>
```

d. Supprimer une donnée

```
<a href="supprimer.php?id=<?php echo $ligne['id']; ?>">Supprimer</a>
```

Fichier `supprimer.php` :

```
<?php
include("connexion.php");
$id = $_GET['id'];

$requete = "DELETE FROM etudiants WHERE id=$id";
if (mysqli_query($connexion, $requete)) {
    echo "Étudiant supprimé !";
} else {
    echo "Erreur : " . mysqli_error($connexion);
}
?>
```

4. Connexion à une base MySQL avec l'extension PHP « PDO »

Exemple :

```
<?php
try {
    $bdd = new PDO('mysql:host=localhost;dbname=ecole;charset=utf8',
'root', '');
    echo "Connexion réussie !";
} catch (PDOException $e) {
    die("Erreur : " . $e->getMessage());
}
?>
```

Explications :

- `PDO` est une **classe**.
- `new PDO(...)` crée un objet de connexion.
- Le bloc `try/catch` intercepte les erreurs éventuelles.

Structure du DSN (Data Source Name) :

```
mysql:host=nom_serveur;dbname=nom_base;charset=utf8
```

a. Exécution de requêtes SQL

Insertion de données

```
<?php
$bdd = new PDO('mysql:host=localhost;dbname=ecole;charset=utf8', 'root',
'');
$requete = "INSERT INTO etudiants(nom, prenom, age) VALUES ('Yao', 'Koffi',
21)";
$bdd->exec($requete);
echo "Nouvel étudiant ajouté.";
?>
```

`exec()` exécute une requête sans retour (INSERT, UPDATE, DELETE).

b) Lecture de données (SELECT)

```
<?php
$bdd = new PDO('mysql:host=localhost;dbname=ecole;charset=utf8', 'root',
'');
$reponse = $bdd->query("SELECT * FROM etudiants");

while ($donnees = $reponse->fetch()) {
    echo $donnees['nom'] . " " . $donnees['prenom'] . " (" .
$donnees['age'] . " ans)<br>";
}
$reponse->closeCursor();
?>
```

`query()` est utilisée pour exécuter une requête de lecture.
`fetch()` renvoie chaque ligne du résultat sous forme de tableau associatif.

c) Mise à jour et suppression

```
// Mise à jour
$bdd->exec("UPDATE etudiants SET age = 22 WHERE nom = 'Yao'");

// Suppression
$bdd->exec("DELETE FROM etudiants WHERE nom = 'Yao'");
```

c. Sécurisation avec les requêtes préparées (prévention des injections SQL)

Exemple :

```
<?php
$bdd = new PDO('mysql:host=localhost;dbname=ecole;charset=utf8', 'root',
'');

$req = $bdd->prepare("INSERT INTO etudiants (nom, prenom, age) VALUES (?,
?, ?)");
```

```
$req->execute(['Kouadio', 'Marie', 23]);

echo "Étudiant ajouté avec succès !";
?>
```

Les requêtes préparées empêchent qu'un utilisateur injecte du code SQL malveillant.

Autre exemple avec bindParam()

```
$req = $bdd->prepare("INSERT INTO etudiants (nom, prenom, age) VALUES
(:nom, :prenom, :age)");
$req->bindParam(':nom', $nom);
$req->bindParam(':prenom', $prenom);
$req->bindParam(':age', $age);

$nom = "N'Da";
$prenom = "Aïcha";
$age = 20;
$req->execute();
```

d. Exemple complet : affichage dynamique

Objectif : afficher une table HTML des étudiants enregistrés.

```
<?php
$bdd = new PDO('mysql:host=localhost;dbname=ecole;charset=utf8', 'root',
'');
$reponse = $bdd->query("SELECT * FROM etudiants");
?>

<table border="1">
    <tr><th>Nom</th><th>Prénom</th><th>Âge</th></tr>
    <?php while ($etudiant = $reponse->fetch()) { ?>
        <tr>
            <td><?= $etudiant['nom'] ?></td>
            <td><?= $etudiant['prenom'] ?></td>
            <td><?= $etudiant['age'] ?></td>
        </tr>
    <?php } ?>
</table>
```

5. Bonnes pratiques et erreurs à éviter

- Toujours **valider les entrées utilisateur** (`filter_input`, `htmlspecialchars`)
- Utiliser les **requêtes préparées** (`mysqli_prepare` ou `PDO`) pour éviter les **injections SQL**
- Ne jamais afficher d'erreurs SQL en production
- Toujours utiliser **les requêtes préparées**.
- Fermer les curseurs après lecture (`$stmt->closeCursor()`).
- Éviter les requêtes SQL dans les boucles.
- Vérifier les retours de fonctions (`false` en cas d'erreur).
- Centraliser la connexion à la BDD dans un fichier séparé `connexion.php` et l'inclure avec `require_once`.